# Roadwatch: A Geospatial Decision-Support Framework for Road-Damage Monitoring and Maintenance Prioritization in Smart Cities

**[Author 1 Name]**
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India

**[Author 2 Name]**
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India

**[Author 3 Name]**
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India

**[Faculty Corresponding Author Name]**
Faculty / Corresponding Author
Vellore Institute of Technology, Chennai, India

*Corresponding author email: [faculty institutional email]* | *Manuscript version: 11 October 2026*

## ABSTRACT

Road-surface defects are spatially distributed infrastructure events that require timely identification, consistent records, and coordinated maintenance decisions. Computer-vision models can identify visible damage in road imagery, yet a detection alone does not create an actionable municipal work item: the observation must be associated with a reliable location, reviewed in context, assigned a workflow state, and made discoverable across a road network. This paper presents Roadwatch, a browser-based geospatial decision-support application for organizing road-defect records and supporting municipal triage. The implementation combines HTML, CSS, and JavaScript with Leaflet for interactive mapping and a CARTO basemap derived from OpenStreetMap. A shared client-side state coordinates a severity-coded map, summary indicators, time-window controls, filters, a sortable detection register, record-level details, and maintenance-status actions. The paper contributes a modular architecture, a normalized defect-event schema, an operational lifecycle, and a data-science extension design for coordinate validation, duplicate suppression, road-segment aggregation, spatial hotspot analysis, and transparent priority ranking. Functional validation focuses on consistency between map, table, filters, summaries, and record actions. A 30-record application registry is characterized to illustrate the summaries available to operators; its stored confidence values are not interpreted as independently measured detector accuracy. The paper also provides a reproducible evaluation protocol for future vision-model and municipal-outcome studies, including precision, recall, mean average precision, geospatial error, duplicate rate, and maintenance response time. Roadwatch is positioned as an inspectable decision-support layer for safer and more data-informed urban road maintenance, with a practical pathway toward connected sensing and persistent municipal systems.

**INDEX TERMS—** geospatial decision support, road damage detection, road asset management, smart cities, spatial analytics, maintenance prioritization.

### PAPER SCOPE

*This manuscript focuses on the geospatial dashboard and maintenance-triage architecture. It does not claim a novel detector or infer real-world maintenance impact from interface functionality. The machine-learning and spatial-statistics sections define a reproducible integration and evaluation plan; reported registry statistics are limited to the 30 records included in the application.*

## NOMENCLATURE

**TABLE N. SYMBOLS AND ABBREVIATIONS**

| Symbol / term | Definition |
|---|---|
| AP / mAP | Average precision / mean average precision for object detection. |
| CRS | Coordinate reference system. |
| GIS | Geographic information system. |
| GPS | Global positioning system. |
| IoU | Intersection over union of predicted and ground-truth boxes. |
| KDE | Kernel density estimation. |
| KPI | Key performance indicator. |
| LISA | Local indicators of spatial association. |
| RDD | Road Damage Dataset. |
| TP / FP / FN | True positives / false positives / false negatives. |

## I. INTRODUCTION

Urban road networks are dynamic public assets. Traffic loading, rainfall, utility cuts, temperature cycles, construction activity, and ordinary material fatigue can produce potholes, cracks, edge failures, and surface deformation. These defects are local in physical extent but network-wide in management consequence: a small defect on a heavily used corridor may deserve earlier attention than a larger defect on a low-volume road, while several repeated defects on the same segment may indicate a deeper maintenance issue. Municipal teams therefore need more than a list of observations. They need a traceable way to locate, inspect, classify, prioritize, assign, and close defects over time.

Traditional inspection practices remain essential, but periodic patrols and manually assembled reports can create latency between observation and action. The record may be stored in a message, photograph, spreadsheet, contractor report, or separate asset register. An observation that lacks a usable coordinate or status can be difficult to verify, compare with nearby reports, or route to the responsible team. The practical problem is consequently both a perception problem and an information-coordination problem.

Computer vision has advanced the perception side of this challenge. Maeda et al. demonstrated road-damage detection and classification from smartphone images, including a large annotated image collection and comparisons of model execution on GPU servers and smartphones [1]. The later RDD2022 dataset extended the geographic diversity of road-damage imagery to six countries, including India, and includes more than 47,000 images and 55,000 annotated instances [2]. A 2025 survey reviews the trade-offs among detection accuracy, model complexity, datasets, and real-time deployment [3]. These studies make automated observation plausible, but a perception output must still be organized into an operational record to affect municipal work.

Roadwatch addresses this operational layer. It is a web-based geospatial dashboard for viewing road-defect records in a real map context and managing the record-level review lifecycle. The application integrates a Leaflet map with a severity-coded marker layer, an attention banner, key performance indicators (KPIs), severity and time-trend summaries, a searchable and sortable table, a detail drawer, and status-changing actions. Shared client-side state ensures that the displayed collection is filtered consistently across the map and table. The design aims to reduce interface fragmentation by bringing record discovery, visual context, and workflow actions into one surface.

The research question is not whether a particular detector outperforms the state of the art. Instead, the paper asks how a lightweight, browser-based geospatial interface can organize road-defect events for municipal triage, and how a reproducible data-science layer can extend that interface toward location-aware prioritization without conflating detector confidence with physical severity or maintenance urgency.

Four contributions are made. First, the paper describes a modular application architecture connecting structured defect records to map, summary, filter, and workflow components. Second, it formalizes a defect-event schema and lifecycle suitable for integration with visual detection, field reports, or third-party systems. Third, it specifies a set of data-science methods—coordinate quality checks, spatiotemporal duplicate grouping, road-segment aggregation, hotspot discovery, and transparent priority scoring—with equations and validation requirements. Fourth, it defines separate evaluation dimensions for interface behavior, object detection, geospatial data quality, and municipal workflow outcomes.

The remainder of the paper is organized as follows. Section II reviews related work. Section III formulates the problem and requirements. Sections IV–VI describe the research method, architecture, record model, and workflow. Section VII details the data-science extension. Section VIII documents implementation. Section IX reports functional and registry-level evaluation. Sections X–XII discuss deployment, validation, risks, and the Smart City context. Section XIII concludes the paper, followed by appendices with schemas and test protocols.

## II. BACKGROUND AND RELATED WORK

Road-damage monitoring draws from computer vision, mobile sensing, geographic information systems (GIS), data quality, and public-asset maintenance. These strands solve related but different subproblems and should be integrated deliberately rather than treated as interchangeable.

### A. Image-Based Road Damage Detection

Maeda et al. formulated road-damage classification as an object-detection problem using images captured by smartphones mounted on vehicles [1]. Their study is an important precedent for low-cost capture because it relies on widely available imaging hardware rather than requiring a

dedicated survey vehicle. Deep neural object detectors can localize multiple visible defects within an image and output a class and confidence score. These outputs are useful for screening, but model behavior varies with illumination, motion blur, weather, camera position, road texture, occlusion, and defect scale.

Later benchmark work expanded the breadth of road-damage data. RDD2022 includes images from Japan, India, the Czech Republic, Norway, the United States, and China, with four damage categories: longitudinal cracks, transverse cracks, alligator cracks, and potholes [2]. The dataset is designed to support model development and comparison across countries and acquisition conditions. The presence of Indian imagery is valuable for a Chennai-oriented system; however, an India label alone does not guarantee that a trained detector generalizes to the roads, cameras, monsoon conditions, or surface materials of a particular municipal zone.

One-stage detectors, including the YOLO family, jointly predict object locations and classes in a single forward pass and are often selected where inference latency matters [6]. Two-stage detectors such as Faster R-CNN generate candidate regions and refine them, offering a different accuracy–latency trade-off [5]. The optimal choice depends on the target hardware, input resolution, class imbalance, detection size, and required processing throughput. A municipal application should compare models under the same data split and deployment conditions instead of adopting a detector name as evidence of suitability.

Abdelwahed et al. review recent real-time road-damage methods and datasets, emphasizing the continuing gap between research models and practical deployments [3]. Their synthesis supports a system-level view: the detection model is one component in a broader pipeline that must account for capture, processing cost, output reliability, and use by downstream operators. Roadwatch therefore treats incoming detections as structured events and keeps the dashboard independent of a specific model framework.

### B. Mobile Sensing and Crowdsourced Observation

Image-based approaches are complemented by mobile-sensor and crowdsourcing strategies. Singh et al. described “Smart Patrolling,” a road-surface monitoring approach using smartphone sensors and crowdsourcing to detect road conditions [4]. Accelerometers, inertial measurements, GPS, and user-submitted observations can supply evidence that is different from the visual appearance of a defect. Sensor-derived anomalies are not equivalent to confirmed potholes: vehicle dynamics, speed, suspension, lane position, and other road irregularities can influence the signal. A robust operational system should therefore retain the observation method and provenance and allow review rather than silently combining incompatible signals.

Crowdsourced observations can expand coverage beyond scheduled inspections, but they introduce duplicate reports, unequal reporting density, varying coordinate accuracy, and potential bias toward roads traversed by contributors. A map makes distribution visible but does not automatically correct these biases. Roadwatch’s record model can represent observations independently of their source, while the proposed data-science layer adds explicit deduplication, source tracking, and segment-level denominators.

### C. Geospatial Decision Support and Spatial Statistics

GIS is well suited to infrastructure operations because road defects are located within a network, not merely scattered across a screen. Spatial aggregation can reveal corridors with repeated observations, clusters of high-priority events, or areas where inspection coverage is weak. Spatial association methods such as local indicators of spatial association can distinguish local patterns from a global average [10], while the Getis–Ord family of statistics supports local hotspot analysis under an explicit spatial-neighborhood definition [9]. Moran’s early work established a foundational statistic for spatial autocorrelation [11]. These methods are not interchangeable: each has assumptions concerning spatial weights, null hypotheses, scale, and the statistical meaning of a cluster.

For road assets, distance alone is not always the right neighborhood. Two defects may be geographically close but lie on separate carriageways or parallel roads, while defects farther apart along a winding road may belong to the same maintenance segment. A mature deployment should combine coordinate proximity with map-matched road-segment identifiers and direction of travel when available. Roadwatch begins with point locations and a visual network context; segment-based aggregation is a defined extension that improves the operational meaning of hotspot outputs.

Spatial analysis must also account for exposure. A corridor with many observations may simply receive more inspection trips or carry more traffic. Counts should therefore be interpreted alongside road length, inspection effort, traffic exposure, and reporting source. Where those denominators are unavailable, the output should be described as an observed-record hotspot rather than a definitive estimate of underlying road-condition risk.

### D. Data Quality and Operational Workflows

Data quality is multi-dimensional. Wang and Strong distinguish consumer-oriented aspects of data quality beyond simple accuracy, including completeness, consistency, and fitness for use [17]. For road maintenance, a syntactically valid coordinate can still be misplaced; a recent timestamp can still use the wrong time zone; and a correct detection can be operationally unusable if no road owner or status is known. Roadwatch’s data contract should therefore preserve values alongside provenance, validation flags, and review history.

Mapping libraries such as Leaflet provide map initialization, markers, popups, and event handling without forcing the application to use a single tile provider [12]. CARTO supplies basemap services derived from OpenStreetMap data, with visible attribution requirements and an API-key mechanism [13], [14]. These technologies reduce the

amount of mapping infrastructure the application must implement itself, while leaving service availability, usage limits, key configuration, and attribution as operational responsibilities.

India's Smart Cities Mission emphasizes core urban infrastructure, quality of life, sustainable environments, and smart solutions [16]. Roadwatch aligns with those objectives at the level of maintenance information coordination. It should not be interpreted as a substitute for engineering inspection or a municipal asset-management policy: it is a decision-support interface that can help staff inspect and organize available evidence. The broader societal link is consistent with United Nations Sustainable Development Goal 11, which includes safer, more inclusive, and sustainable cities and transport systems [15].

### E. Research Gap and Positioning

The reviewed work provides strong building blocks for image detection, sensing, datasets, spatial analysis, and web mapping. The integration gap addressed by Roadwatch is narrower and practical: connecting an interpretable record schema to a shared geospatial view and a maintenance lifecycle, while defining how future analytics should be evaluated. Prior detection research primarily evaluates perception quality; mapping applications can display points but may not integrate review and repair states; and maintenance workflows can track tasks without exposing spatial context in a unified dashboard. Roadwatch focuses on this intersection. Its contribution is an implementation and an explicit integration blueprint, rather than a claim of a new object detector or a new spatial statistic.

## III. PROBLEM FORMULATION AND RESEARCH QUESTIONS

Let a road-defect observation be represented by an event $e_i$ with an identifier, an observation time, a geographic position, one or more damage attributes, a confidence value when available, a workflow status, and evidence provenance. The operational objective is to make each valid event discoverable in spatial and tabular views and to support consistent transition from observation to review, scheduling, closure, or dismissal. A second objective is to allow multiple event sources and analytics modules to feed the same decision-support layer without requiring the user interface to be rewritten for each detector or data provider.

The study is guided by four research questions: RQ1: How can a lightweight browser application synchronize geospatial records and maintenance workflow state across map and tabular views? RQ2: Which record fields and validation rules are required to make road-defect observations auditable and spatially usable? RQ3: How can data-science methods improve prioritization while keeping assumptions, weights, and uncertainty visible? RQ4: Which evaluation dimensions are necessary to assess an integrated road-monitoring tool without conflating software functionality, detector accuracy, geospatial data quality, and municipal outcomes?

The scope is deliberately stratified. The current application layer provides browser-based visualization, record discovery, summaries, and state actions on structured records. The analytics layer specifies reproducible methods for event normalization, duplicate grouping, hotspot analysis, and ranking. A connected vision system may generate events, but the quality of the model must be measured in an independent experiment with annotated imagery. A functioning map or a high displayed confidence number cannot substitute for such a benchmark.

The unit of observation is a road-defect record, whereas the unit of maintenance decision may be a record, a cluster of records, or a road segment. This distinction is central. A pothole detector may return several bounding boxes for one physical defect across consecutive frames; treating these as separate maintenance jobs inflates workload. Conversely, merging distinct defects located near one another may obscure severity and repair scope. The system must preserve raw events and make aggregation decisions traceable.

## IV. RESEARCH METHOD

This work follows an engineering-oriented design cycle: characterize the operational problem, define record and interface requirements, implement a browser-based decision-support layer, inspect its behavior through functional test scenarios, and derive a modular analytics specification from the related literature. The method is appropriate to a software artifact whose primary contribution is integration and usability rather than a new visual-recognition algorithm.

Requirements were organized around seven activities: discover a record, locate it on a map, compare severity and status, restrict the time window, search the register, inspect details, and change the record's workflow state. Cross-cutting requirements include consistency between components, responsive layout, visible basemap attribution, clear severity/status language, keyboard access for primary controls, and graceful behavior when external map resources are unavailable.

The implementation is examined at two levels. At the interface level, test cases cover map initialization, marker selection, filtering, search, sorting, summaries, detail rendering, status changes, theme behavior, and layout adaptation. At the registry level, the current 30 records are summarized to check whether the dashboard's aggregations are interpretable. This descriptive summary is not an estimate of citywide pothole prevalence and is not used to infer detector accuracy. For machine-learning extensions, a separate benchmark is specified using annotated image data and held-out evaluation splits.

The paper uses three evidence classes. First, implementation facts are derived from the application source, including the technologies, record fields, filter rules, and available actions. Second, published data and methods are attributed to the literature. Third, the proposed analytical formulas and evaluation protocol are design recommendations that

require calibration and field validation before they should drive real dispatch decisions. Keeping these classes distinct makes the manuscript reproducible and prevents an architectural proposal from being confused with an empirical finding.

## V. ROADWATCH SYSTEM ARCHITECTURE

Roadwatch is organized as a browser application with a presentation layer, shared client-side state, structured records, map services, and an operator workflow. The single-document implementation contains the HTML layout, CSS design tokens, record definitions, filtering helpers, render functions, and interaction handlers. This structure supports rapid deployment and reduces setup overhead, while a future production environment can move record services and state transitions to a backend without changing the conceptual interface boundaries.

The presentation layer uses HTML for structure, CSS for the responsive layout and theme system, and vanilla JavaScript for state and rendering. Leaflet provides the map object, tile layer, circle markers, popups, map events, and size invalidation [12]. The base layer uses CARTO's raster basemap service, which requires an API key and visible attribution to both CARTO and OpenStreetMap [13], [14]. A client should restrict a key to intended web origins when available, monitor request quotas, and never treat a browser-visible basemap key as a secret capable of authorizing privileged data operations.

The dashboard contains five tightly coordinated functional zones: (1) a critical-defect attention banner; (2) KPI cards; (3) the interactive defect map; (4) severity and time-trend summaries; and (5) a detection table connected to a detail drawer. The same record object feeds these zones. This reduces disagreement between the count shown in a table and the marker displayed on the map, provided each rendering function uses the same range, filter, and search state.

The system state includes the selected time range, active filter chip, search query, selected record identifier, sort key, and sort direction. The selected time range scopes records into 24-hour, 7-day, or 30-day windows. Status/severity filters and the text query refine the visible list. The table applies the selected sort key to severity, confidence, or detection time, while the map renders the filtered record collection. Selecting a marker or row opens the detail drawer for the same identifier.

The data contract separates record attributes from rendering. A record includes an ID, street, area, latitude, longitude, severity, confidence, detection time, status, assigned crew, and configured physical attributes. The design can accommodate optional source metadata, image references, detector version, coordinate accuracy, and repair evidence. Optional fields should be added with schema versioning so that new integrations do not silently alter the meaning of existing records.

Figure 1 presents the architecture. The implemented browser layer consumes structured records and exposes the operator workflows. The map-tile service provides background cartography. Service interfaces can be added for ingestion, persistent storage, authentication, and analytics. For a production deployment, persistent records should live behind an API rather than in a client-local array; the browser should remain a user interface, not the authority for audit history or access control.

The architecture also follows a separation-of-concerns principle. A detection engine answers "what may be visible in this frame?" A geospatial registry answers "where and when was the observation made, and can it be trusted?" The workflow layer answers "what action has been taken?" The analytics layer answers "which events or segments deserve attention under the current policy?" This prevents a model's confidence score from being misused as a direct measure of repair urgency.

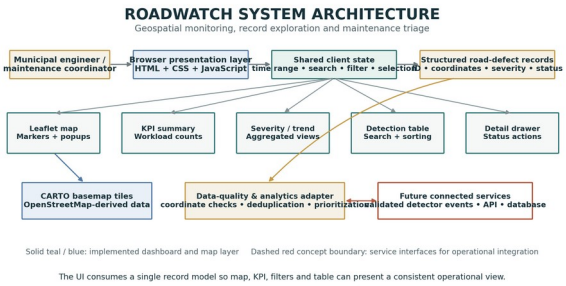



*Fig. 1. Roadwatch layered architecture. The browser dashboard and map layer consume structured defect records; ingestion, analytics, and persistence are modular service interfaces.*

### A. Basemap and Map Rendering

The map is initialized around Chennai at a city-level zoom and displays each record as a circle marker. Marker color reflects the configured severity class, marker radius varies by severity, and repaired records use an outline style. A popup summarizes the record identifier, road, area, status, and confidence field. Marker selection opens the detail drawer. These visual mappings help an operator scan urgency without opening each record, while the popup and table provide text alternatives to color alone.

The map tile layer and the defect marker layer are separate. This separation allows the basemap to be changed without modifying business logic and permits markers to be refreshed independently of the underlying map. The displayed marker location is derived from the record's latitude and longitude, so coordinate validation must occur before a record is accepted into the operational dataset. A range check can reject obvious malformed values, but cannot prove that a point sits on the correct carriageway; map matching and field verification are stronger checks.

Leaflet requires the map container to have a defined height, and dynamic layouts may need a call to invalidate the map size after container changes [12]. Roadwatch invokes a resize refresh so that the map can adapt when the viewport or surrounding layout changes. CARTO basemap attribution is retained in the map control. OpenStreetMap's public tile service has a separate usage policy; applications should not

assume its tile endpoint is an unrestricted production CDN [14].

### B. Dashboard State and Summary Views

The KPI area summarizes records needing attention, records in review, scheduled work, repaired work, and total records in the selected period. The critical-attention banner draws attention to high-severity records that are still new or in review. The severity-mix view counts records by severity after excluding dismissed items. The trend view buckets records into time intervals and the mean-confidence label calculates the arithmetic mean of the confidence fields in the scoped collection.

The summaries are intentionally descriptive. A count of “repaired” is a workflow-state count rather than independent proof of road condition after repair. Mean confidence represents the values carried by records, not a calibrated probability of correctness unless calibration has been established against labeled data. A detection-volume chart can indicate changes in recorded events but cannot, by itself, distinguish a true rise in road damage from a rise in inspection coverage or system usage. Labels and documentation should preserve these distinctions.

The application uses filters for all records, needs attention, critical, scheduled, and repaired states. Search matches the road name, area, or record identifier. Status transitions are presented contextually: new or in-review records can be scheduled; scheduled records can be marked repaired; repaired records can be reopened; and unresolved records can be dismissed as false positives. These actions make the UI a workflow surface rather than only a visualization.

### C. Integration Boundaries

A connected implementation should define a stable ingestion contract and make external services replaceable. The minimum event endpoint would accept a schema-versioned record and return a persistent event identifier. Server validation should check required fields, coordinate bounds, timestamp format, confidence ranges, allowed status values, source identity, and duplicate identifiers. The database should record both current state and state-transition history. API authentication, role-based access, rate limits, and audit logging belong on the server side.

A data adapter should transform model-specific outputs into the shared event schema. For example, an object detector may return one or more bounding boxes, class labels, and confidence values per image. An edge device can attach a timestamp, approximate GPS fix, device identifier, and image reference. The adapter should not invent a coordinate when GPS is unavailable; instead, it should store the event with a location-quality state such as `pending_geolocation`. Once valid, the event becomes visible in the map according to the application’s data-quality policy.

Persistent storage becomes important when several operators use the application concurrently. Without server-authoritative state, one user’s update may not be visible to another, and a browser refresh can restore default values. A production backend should use transactional updates, version conflicts or optimistic concurrency, idempotent ingestion, and immutable audit events. The proposed architecture does not bind to a particular backend; relational databases with geospatial extensions are a practical option for structured records and spatial queries.

## VI. DEFECT DATA MODEL AND OPERATIONAL WORKFLOW

The basic event schema should be compact enough for ingestion while preserving enough metadata for inspection and analysis. Table I summarizes essential and recommended fields. Existing front-end records include the core identification, road, area, coordinate, severity, confidence, time, status, crew, and physical-attribute fields. The additional provenance fields make the schema suitable for integration with real cameras, models, APIs, and repair verification.

An event ID should be stable across re-rendering and should not encode a secret or a user’s identity. `observed_at` should use an explicit standard timestamp, preferably UTC stored with an offset or converted to UTC at ingestion. `latitude` and `longitude` should use WGS 84 decimal degrees unless the system documents another reference system. `severity` should be a controlled class with an associated severity-policy version. `status` should be constrained to a documented set of transitions. An unknown or unreviewed field should be null or represented with a specific quality flag rather than silently assigned a plausible number.

The severity scale can use four ordered classes—minor, moderate, severe, and critical—to support scanning and initial triage. Severity is not equivalent to detector confidence. Confidence describes a model’s score under its trained output semantics; severity describes the apparent or assessed seriousness of the road condition. They may be correlated in some data, but they answer different questions and should occupy separate fields. A defect with a high-confidence minor crack should not automatically outrank a lower-confidence but clearly critical pothole without an explicit decision policy.

The lifecycle is a controlled state machine. A new record can move to in-review, scheduled, or dismissed after review; a scheduled record can move to repaired; a repaired record can be reopened if inspection finds the problem persists; and a dismissed record may be restored when new evidence appears. The UI exposes the practical actions needed to enact these transitions. A connected service should validate transitions on the server, record the actor and timestamp, and require verification evidence before definitive closure where local policy requires it.

Figure 2 shows the broader event lifecycle from observation to repair verification. Only after data quality and duplicate checks should a detected event be considered an independent work item. A workflow state is a record of administrative action; it is not a physical measurement. For example, a record should not be labeled repaired solely

because a status button was pressed if operational policy requires a crew report or field confirmation. The data model should allow both a maintenance status and a verification status so that the organization can distinguish task completion from quality assurance.

**TABLE I. ROAD-DEFECT EVENT SCHEMA**

| Field | Type / example | Purpose and validation |
|---|---|---|
| event_id | String; PH-2098 | Stable unique identifier; reject duplicate IDs. |
| observed_at | ISO 8601 timestamp | Capture time; normalize time zone and preserve source time. |
| street, area | String values | Human-readable road and locality; normalize spellings. |
| latitude, longitude | WGS 84 decimal degrees | Map placement; range check plus road/field verification. |
| damage_class | Controlled label | Pothole, longitudinal crack, transverse crack, alligator crack, or local class. |
| severity | Minor / moderate / severe / critical | Triage category with policy version and review status. |
| confidence | Floating-point score, 0–1 | Store only with model/version provenance; do not treat as calibrated probability by default. |
| status | New / review / scheduled / repaired / dismissed | Workflow state; validated transition history. |
| source | Camera / survey / citizen / inspector | Observation origin, device or submission identifier. |
| geometry_quality | GPS accuracy or quality flag | Supports location QA and exclusion from segment analytics. |
| evidence_ref | Image URI / report attachment | Links the record to evidence under access and retention policies. |
| crew, timestamps | Assignment and status history | Accountability, response-time measurement, and reopen analysis. |

### A. Coordinate Validation and Map Matching

Coordinate validation has at least four levels. Syntax validation confirms numeric latitude and longitude values. Geographic-range validation rejects points outside the intended operating region. Plausibility validation checks whether the point falls close enough to a mapped road segment for the stated observation method. Operational validation checks whether the point identifies the correct side, direction, access lane, and municipal jurisdiction. Range checks alone are insufficient; the user may need to inspect a map or receive a GPS quality estimate.

GPS uncertainty should be stored separately from the coordinates. A receiver may report a point with horizontal accuracy of several metres, and buildings, tree cover, multipath, or device placement can degrade precision. When map matching is available, the system should preserve the original point, matched road segment, distance to segment, and match confidence. This audit trail allows later correction without losing the source observation. Low-quality records can remain visible with a warning while being excluded from automatic dispatch scoring.

### B. Duplicate and Repeat-Defect Handling

Duplicate handling must distinguish duplicate observations of the same physical defect from genuinely distinct defects on the same road. A first-pass candidate match can use time separation, geographic distance, road segment, damage class, and image similarity if available. A record pair within a chosen distance and time window may be flagged for review, but should not automatically be deleted. Two potholes can occur within a few metres, and a single pothole can be captured repeatedly from slightly different GPS positions.

The recommended pattern is soft deduplication: keep every raw observation, group likely repeats into a canonical incident, and retain links from each observation to the group. Group-level fields can include first observed time, last seen time, observation count, source diversity, severity history, and verification state. This supports recurring-condition analysis while preventing a patrol that repeatedly traverses a road from creating one repair job per frame.

## VII. DATA-SCIENCE EXTENSION FOR PRIORITIZATION

The data-science layer extends a map of individual detections into a reproducible municipal decision process. It does not require every analytical module to run in the browser. Event normalization, geospatial joins, deduplication, statistics, and model inference are better suited to a service layer once data volume and multi-user operations grow. Roadwatch's role is to make the resulting evidence interpretable: show the observation, locate it, expose its quality flags, explain the rank, and allow a human operator to make the final decision.

Four principles guide the design. First, preserve raw observations even when records are grouped. Second, keep detection confidence, assessed severity, location confidence, and maintenance priority separate. Third, expose the variables that contribute to an analytical recommendation. Fourth, calibrate any weights or thresholds against local data and municipal policy. A composite score is a decision aid rather than an objective truth, and its output should not override safety rules or engineering inspection.

### A. Event Normalization and Quality Gates

Let raw observation $r_i$ be converted to a normalized event $e_i$ by a deterministic transformation $N(\cdot)$: $e_i = N(r_i, v)$, where $v$ denotes the schema version. Normalization standardizes fields, maps model-specific labels to canonical labels, converts timestamps, checks coordinate ranges, and attaches source metadata. Invalid records should carry explicit error codes and remain available for correction. Silent coercion—for example, replacing a missing coordinate with the city centre or converting an unknown severity to "minor"—creates dangerous false precision.

Quality gates can assign a quality vector $q_i$ = [q_geo, q_time, q_class, q_source, q_evidence], where each

component represents a validation outcome. The vector need not be reduced to a single score. A dispatch policy may require verified coordinates and a valid status history even when a vision model is confident. The interface can display a location-quality badge or prevent automatic priority ranking when essential fields are missing. The acceptance rules should be configurable because a public complaint, a high-precision camera, and an inspector-confirmed finding have different evidence strength.

Time consistency is also important for trends. The interface may present “hours ago” from an observation timestamp, but analytics should retain canonical timestamps and compute the display string at view time. If records are backfilled, the detection volume should use event time rather than ingestion time, while latency metrics should measure both. Separating `observed_at`, `received_at`, `reviewed_at`, `scheduled_at`, and `closed_at` makes response-time calculations interpretable.

### B. Duplicate Candidate Scoring

For observations i and j, let $d_{ij}$ be the geodesic distance, $\Delta t_{ij}$ the absolute time gap, $s_{ij}$ a same-segment indicator, $c_{ij}$ a class-compatibility term, and $v_{ij}$ an optional image-embedding similarity. A transparent pairwise candidate rule is:

$$D(i,j) = 1[d_{ij} \le \tau d] \cdot 1[\Delta t_{ij} \le \tau t] \cdot 1[s_{ij} = 1] \cdot C(i,j)$$

*Here $\tau d$ and $\tau t$ are locally calibrated distance and time thresholds. $C(i,j)$ is a class-compatibility rule or similarity score. The indicator is a candidate-generation mechanism, not an automatic merge decision. Suggested initial settings must be tuned against manually labeled pairs and should remain configurable by road type and sensor accuracy.*

### C. Haversine Distance and Spatial Thresholding

Geodesic distance can be computed with the Haversine formula. Let latitude and longitude be expressed in radians, $\Delta\varphi = \varphi_2 - \varphi_1$ and $\Delta\lambda = \lambda_2 - \lambda_1$; with Earth mean radius R, distance is estimated as below. For short urban distances, local projected coordinates may also be used where an appropriate projection is available.

$$a = \sin^2(\Delta\varphi/2) + \cos(\varphi_1)\cos(\varphi_2)\sin^2(\Delta\lambda/2); \quad d = 2R \operatorname{asin}(\sqrt{a})$$

*R is the selected Earth radius in metres (approximately $6.371 \times 10^6$ m for a spherical approximation). The computation estimates distance between two points; it does not provide road-network distance or identify the correct carriageway.*

### D. Road-Segment Aggregation and Hotspot Discovery

Point counts are easy to compute but can be misleading when survey effort differs across the network. Where a road-segment inventory is available, each event can be assigned to a segment identifier, and metrics can be normalized by length, inspection passes, vehicle-kilometres, or traffic volume. A basic segment rate is $\lambda_k = n_k / L_k$, where $n_k$ is the count of eligible incidents on segment k and $L_k$ is segment length in kilometres. When inspection effort is known, $n_k$ can also be divided by the number of observation passes. Denominators must be named explicitly; “defects per kilometre” and “defects per inspection pass” answer different questions.

Kernel density estimation (KDE) can provide a smoothed visual surface over point events. A conceptual form is $\lambda(s) = \Sigma_i\, w_i\, K(d(s,s_i)/h)/h^2$, where s is a map location, $s_i$ is an observation, $w_i$ is a documented event weight, K is a kernel function, and h is the bandwidth. In a road-network environment, a network-constrained KDE is preferable to Euclidean smoothing when the goal is to highlight defects along roads rather than nearby off-road space. The bandwidth should correspond to a meaningful operational scale and should be tested for sensitivity.

Local spatial statistics may be used when the analysis requires statistical hotspot evidence rather than only a heatmap. Getis–Ord Gi* compares the local weighted sum with the global distribution under specified spatial weights [9], while LISA statistics identify local clusters and spatial outliers [10]. These analyses require a declared neighborhood, null model, and multiple-testing policy. A colored hotspot map without these details should be described as a density visualization, not as a statistically significant cluster.

Roadwatch can expose multiple map layers: individual incidents, canonical incident groups, rates by road segment, and hotspot classes. Users should be able to switch among them and inspect the underlying count and denominator. A hotspot with many observations but little survey exposure could be more meaningful than an equally sized cluster on a heavily patrolled corridor. Conversely, sparse observations may reflect low coverage rather than good road condition. A coverage layer is therefore a valuable companion to the defect layer.

### E. Transparent Maintenance Priority Score

A composite score can help order the work queue, provided it is transparent and policy-controlled. One candidate formulation is:

$$P_i = w_s S_i + w_e E_i + w_r R_i + w_a A_i + w_u U_i, \quad \Sigma_j w_j = 1$$

*$P_i$ is a normalized priority score; $S_i$ is assessed severity; $E_i$ is exposure or road criticality; $R_i$ is recurrence evidence; $A_i$ is age or time awaiting action; and $U_i$ is uncertainty or evidence quality. The weights $w_j$ are policy parameters, not empirically learned values in this paper. Uncertainty can be handled as a separate review queue rather than increasing urgency directly.*

### E. Transparent Maintenance Priority Score

Each component must be normalized to a common scale and documented. For example, severity may map the four classes onto an ordered scale from 0 to 1; age may saturate after a policy-defined waiting period; recurrence may use capped unique-observation counts; exposure may use an approved road hierarchy or traffic measure. Uncertainty should not always be added to priority: low certainty may call for verification, while a high-severity potential hazard may demand precautionary review. The policy should therefore support both an urgent-work ranking and an evidence-review queue.

Weights should be selected with municipal stakeholders, tested against retrospective cases, and subjected to sensitivity analysis. A simple baseline that sorts by severity

and age provides a useful comparator. If the composite score does not improve ranking agreement, triage time, or resource allocation over that baseline, complexity is not justified. The user interface should show the score components, not only a single opaque number. Any override should record the reason and actor.

### F. Model and Data-Science Evaluation

For object detection, predictions must be compared with annotated ground truth at a defined intersection-over-union (IoU) threshold. Precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is the harmonic mean of precision and recall. Average precision summarizes a precision–recall curve for a class; mean average precision averages AP across classes or thresholds according to the stated protocol [8]. The threshold convention must be specified because mAP@0.50 and mAP@0.50:0.95 quantify different levels of localization strictness. Detection evaluation should report per-class support because rare damage types may be hidden by aggregate scores.

Inference latency should be reported along with preprocessing and postprocessing time, the device used, image resolution, batch size, and whether timing includes image transfer. The application's downstream workflow adds latency for geolocation, schema validation, network communication, and rendering. A "real-time" claim should therefore define an end-to-end latency target and test it on representative hardware rather than quoting a model-only benchmark.

The data split is critical. Images from the same drive, road segment, video sequence, or repeated capture session should not be scattered across training and test sets when that leads to near-duplicate leakage. A rigorous split groups related captures by route or session; a stronger generalization test can hold out local districts or acquisition conditions. RDD2022 offers geographic breadth [2], but a local validation set is still needed to test transfer to Chennai-specific conditions. Any fine-tuning should document source, license, label mapping, augmentation, class balance, random seed, training configuration, and checkpoint selection.

The evaluation is multi-layered. Detector precision does not measure the quality of GPS coordinates; coordinate accuracy does not measure whether two reports refer to the same physical defect; and a high-priority ranking does not prove that repair teams acted faster. Figure 5 summarizes these independent axes. The proposed evaluation protocol deliberately prevents one successful metric from being used as evidence for unrelated outcomes.

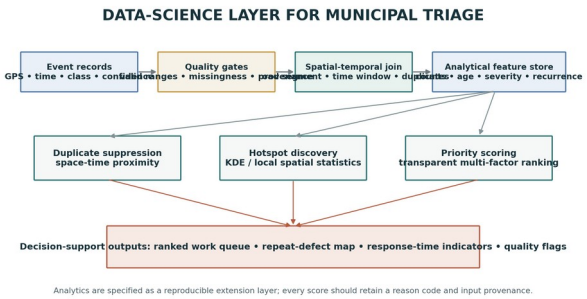



*Fig. 3. Data-science extension architecture from normalized events through quality gates, spatial-temporal processing, analytics and decision-support outputs.*

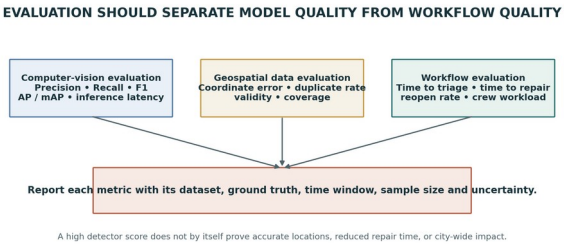



*Fig. 4. Separate evaluation domains for perception, geospatial data and operational workflow.*

## VIII. IMPLEMENTATION DETAILS

The application is implemented as a single HTML file with embedded CSS and JavaScript, with external Leaflet CSS and JavaScript dependencies. This packaging allows local execution with a static web server. The interface does not require a frontend build pipeline. This is suitable for rapid demonstration and controlled pilot use; a multi-operator municipal deployment would benefit from splitting code into modules, adding automated tests, and configuring a backend API.

The CSS uses design tokens for surface colors, borders, typography, severity colors, status colors, spacing, and responsive breakpoints. A sidebar and topbar organize navigation and controls. The layout switches from a two-column application shell to a single-column layout on narrower screens; the map height is adjusted at tablet and phone widths. Theme handling supports light, dark, and system-responsive styling. The selected theme is stored in browser local storage, which is appropriate for a user preference but should not be used for authoritative application records.

The core record collection is an array of structured JavaScript objects. The constructor validates that latitude and longitude are numeric and inside the configured geographic bounds before a record is returned. This catches obvious coordinate errors early. It does not prove that a point is on the correct named road, so a production ingestion endpoint should add road-distance and map-matching checks. Each record includes an identifier, street name, area, latitude and longitude, severity, confidence value, relative timestamp, status, optional crew, and depth/width attributes.

The render orchestration calls the KPI, banner, severity mix, trend, filter chips, map, and table render functions from a single `renderAll` path. This creates a clear update sequence

after a state change. Map and table selection both resolve a stable record ID and open the same detail drawer. Event delegation on the table reduces the number of individual click handlers. Keyboard handling allows Enter to open a focused row and Escape to close the drawer.

Time-range filtering is implemented from an application reference time and a rolling duration map for 24 hours, 7 days, and 30 days. In the record registry, event age is stored as an offset in hours when the application initializes. The "time ago" text is derived from the timestamp for display. For persistent real-world data, timestamps must be stored as actual UTC instants; display labels should be generated from the viewer's locale and never stored as canonical data.

Sorting is applied to the currently visible table rows. Severity sorting uses an explicit rank mapping, confidence sorting uses the stored numeric field, and time sorting uses the event timestamp. Status transitions are executed through a central action function, which updates the record, assigns a crew when scheduling an unassigned record, shows a toast, and rerenders the dashboard. In a production design, status changes should be sent to a server and committed to an audit history instead of remaining only in memory.

The map tile layer is initialized at a Chennai-centred view using CARTO's Voyager raster tiles. Visible attribution references OpenStreetMap and CARTO, and the code checks whether the Leaflet library has loaded before initializing. The map's size is refreshed after layout changes. A robust deployment should also surface asynchronous tile errors, watch API usage, restrict the key to the approved domains, and provide a tile-provider fallback or offline operating procedure based on organizational requirements.

The detail drawer displays severity, status, location, detection age, configured physical fields, and crew assignment. The drawer's actions are state-specific and include schedule, mark repaired, reopen, dismiss, and restore. The illustration in the detail pane is an interface representation associated with the record; it should not be treated as the underlying photographic evidence. An operational event record should instead link to the captured image or inspection attachment and preserve its integrity and access permissions.

For extensibility, a later codebase can separate the user interface into components, the record state into a store, and external services into API modules. Type definitions or schema validation can formalize record contracts. Automated unit tests should cover pure functions such as filtering, sorting, severity ranking, and date bucketing; integration tests should verify marker/table identity and status-transition consistency. Browser-level tests can then confirm tile initialization, responsive layout, and accessible controls.

## IX. EVALUATION AND RESULTS

The evaluation is split into implementation-level functional checks and descriptive registry analysis. This avoids treating UI validation as a computer-vision benchmark. The reported functional review covers the user-visible interactions available in Roadwatch. The registry analysis computes summaries from the 30 records currently loaded by the application. These figures characterize the application's current record distribution and the dashboard's aggregation behavior; they do not estimate the rate of road damage across Chennai.

The functional validation exercised the core paths: initialization of the map and marker layer, record popup and detail selection, time-window selection, search by ID/road/area, filter chips, table sorting, KPI updates, severity distribution, time trend, detail drawer actions, theme toggle, mobile-layout behavior, and map attribution. The checks passed in the project's tested browser workflow. The checklist in Table II provides repeatable steps for regression testing after code changes. Because the checks are manual UI tests rather than an automated browser suite, they establish practical functionality under the test conditions, not exhaustive coverage of all browsers and network conditions.

The registry contains 30 records. By severity, 5 are minor, 11 moderate, 8 severe, and 6 critical. By workflow state, 5 are new, 4 in review, 7 scheduled, 12 repaired, and 2 dismissed. The mean of the 30 stored confidence fields is 0.839. The seven-day window contains 20 records under the application's relative-time model; five records fall within the 24-hour window. The selected-range filters and summary cards are designed to recompute from this scoped collection.

The confidence aggregate requires careful interpretation. The arithmetic mean of stored scores is not the same as calibrated model accuracy, and it does not state what proportion of detections are correct. Without a labeled evaluation set, confidence thresholds, score calibration, and class-specific error analysis, these values should be interpreted only as fields attached to the current registry. The application exposes them for sorting and summary purposes; the scientific evaluation of a future detector remains a separate experiment.

Figure 4 visualizes the registry mix. The repaired share is 12/30, or 40%; scheduled items are 7/30, or 23.3%; new items are 5/30, or 16.7%; in-review items are 4/30, or 13.3%; and dismissed items are 2/30, or 6.7%. These shares can be used to test the aggregation logic and dashboard readability. They must not be interpreted as municipal repair effectiveness because the registry does not encode the sampling mechanism, defect discovery process, or operational denominator required for such a claim.

The interface-level contribution is therefore demonstrated by the consistency of the views and the availability of the record lifecycle. The strongest next empirical step is a controlled usability and workflow study: recruit municipal or domain users, define representative tasks, measure task completion time and errors, and compare the unified dashboard with the users' baseline reporting workflow. A later field study should instrument the entire chain from detection timestamp to verification timestamp and compare

the result against a pre-defined baseline period or matched corridors.

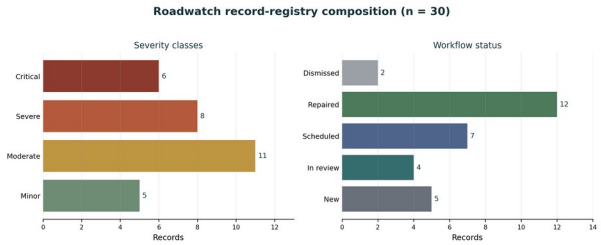



*Fig. 5. Severity and status composition of the 30 records in the current Roadwatch application registry.*

**TABLE II. FUNCTIONAL VALIDATION MATRIX**

| ID | Test scenario | Expected behavior | Result |
|---|---|---|---|
| F01 | Load application and map | Map layer loads; attribution remains visible. | Pass |
| F02 | Select a map marker | Popup identifies the selected record; details open for the same ID. | Pass |
| F03 | Select a table row | Detail drawer opens for the same record. | Pass |
| F04 | Search by ID, road, or area | Map and table reflect the text query. | Pass |
| F05 | Change time window | Scoped KPIs, trend, map, and table update consistently. | Pass |
| F06 | Apply status/severity chip | Visible collection updates without changing record identity. | Pass |
| F07 | Sort severity/confidence/time | Table order follows the selected key and direction. | Pass |
| F08 | Schedule a record | State changes to scheduled and an unassigned crew is selected. | Pass |
| F09 | Mark repaired / reopen | Status action is reflected in the drawer, table, and summaries. | Pass |
| F10 | Dismiss / restore record | Record status updates using the available action. | Pass |
| F11 | Switch theme | Theme updates and user preference is retained in the browser. | Pass |
| F12 | Resize to smaller viewport | Map and application layout adapt to responsive breakpoints. | Pass |

## A. Interpretation of the Functional Results

The passed checks support the claim that Roadwatch functions as an integrated front-end record-exploration and triage interface under the tested setup. The common ID selection mechanism reduces the chance that the map opens a different record from the selected table row. The single state-driven render path helps maintain consistent filtering after user actions. The responsive rules and map-size refresh address a common class of mapping-interface defects: a map container can be present but render with incorrect dimensions after layout changes.

The checks do not establish the availability of external tile services under all network conditions, persistent multi-user consistency, or long-term data durability. These concerns depend on external services and deployment architecture and should be covered by operational tests. The basemap attribution and API usage controls are part of the technical evaluation because a visual map is only sustainable when its provider's terms are observed [13], [14].

The state transition behavior warrants an additional transactional test in a connected deployment. A UI can correctly reflect an action during a single session while a refresh or second operator reveals the absence of server persistence. A production acceptance test should therefore create a record, change its state, reload the page, query it from a second session, and inspect the audit history. This is a deployment validation requirement rather than a criticism of the front-end interaction model.

## B. Recommended Quantitative Experiment

A reproducible road-damage experiment should use an annotated dataset such as RDD2022 [2] and a fixed baseline detector. The experiment should define the class mapping, data split by route/session, image resolution, training epochs, optimizer, augmentations, confidence threshold, non-maximum suppression settings, and inference hardware. At least one held-out test set should represent conditions not seen in training. A local Chennai validation set should be collected or annotated if the intended deployment includes specific camera hardware and road conditions.

The primary detector outcomes should include precision, recall, F1, AP per class, mAP under stated IoU thresholds, false positives per image or kilometre, and end-to-end latency. Precision and recall should be reported with support counts and, where possible, confidence intervals or bootstrap estimates. Model comparisons should keep the test set and operating threshold fixed. When false positives and false negatives have different municipal costs, threshold selection should document the chosen cost trade-off.

Geospatial and workflow outcomes require separate ground truth. A sample of events should be compared with surveyed coordinates or verified map points to estimate location error. Duplicate grouping should be evaluated against manually labeled same-incident pairs. Priority ranking can be assessed through expert ranking agreement, top-k recall of incidents judged urgent, and sensitivity to weights. Operational impact can be measured as median time to first review, median time to schedule, median time to verified repair, reopen rate, and the proportion of overdue events. These studies need baseline data and clear event definitions before any improvement claim is justified.

# X. DISCUSSION

The main practical insight is that road-damage monitoring should be designed as an end-to-end information workflow, not only as an image classifier or a map with colored pins. A municipal operator needs to inspect what was observed, understand where it occurred, compare it with other records,

determine whether it requires action, assign responsibility, and retain the outcome. Roadwatch places these actions in one browser interface and uses a shared record model to coordinate the map, table, summary cards, and status drawer.

The user interface is deliberately lightweight. It can be opened with a static web server, depends on common browser technologies, and delegates map rendering to a provider-agnostic library. This lowers the implementation barrier for a small proof-of-concept or pilot, while making the service boundaries visible for later scaling. The same lightweight structure also makes the limitations easy to reason about: client-side arrays are not substitutes for authoritative shared storage, and visual summaries depend on the quality of the incoming record source.

The data-science methods proposed here are deliberately interpretable. Duplicate matching, severity-age scoring, and segment rates can be inspected by operators. Local spatial statistics can be added when the data volume, road network, and sampling mechanism justify them. An interpretable baseline is a better starting point than a complex model whose ranking cannot be explained. Data-driven weights may become useful later, but training a model to reproduce historical dispatch decisions can also learn past resource-allocation bias; policy constraints and human oversight remain necessary.

There is a potential reinforcing loop in maintenance data. Areas that receive more inspection generate more records, which can trigger more repairs and more repeat inspection, while areas with limited coverage may appear healthy. Coverage and source metadata allow analysts to identify this issue. The system should expose the number of inspections or passes, not only defects. A dashboard that counts only observed failures risks mistaking missing observations for safe infrastructure.

Roadwatch is relevant to smart-city goals because it can improve the legibility and traceability of a municipal workflow. It may help teams locate records, compare priority, coordinate repair, and learn from recurring defects. Benefits such as reduced crashes, lower vehicle operating cost, or faster repair should be treated as evaluation hypotheses until measured. A good next-step field study can test the causal chain from improved visibility to faster review and then to verified maintenance outcome.

## XI. SECURITY, PRIVACY, AND DATA GOVERNANCE

Road-defect records can contain more than coordinates. Images may show vehicle registration plates, faces, pedestrians, homes, or commercially sensitive activity. A capture pipeline should define purpose limitation, collection boundaries, retention periods, access rights, and masking requirements. Image references should not be publicly accessible by default. A public-facing transparency page can communicate how observations are collected and how residents can report or dispute an incident.

Client-visible configuration should be distinguished from secrets. A basemap API key used in a browser must be treated as an origin-restricted public identifier, not as a server credential. CARTO supports API keys and requires visible attribution to CARTO and OpenStreetMap [13]. Privileged data API credentials, signing secrets, database passwords, and administrative tokens must remain server-side. Production configuration should restrict allowed referers, rotate keys when exposed outside policy, monitor usage, and retain an incident response procedure.

Authorization should follow roles. An inspector may create or validate an observation; a municipal reviewer may schedule it; a contractor may update repair progress; and a supervisor may approve closure or reopen an item. These roles should be enforced by backend authorization checks, not solely by hiding buttons in the browser. Every state transition should store actor, timestamp, before/after state, reason code, and related evidence reference. An append-only audit log supports dispute resolution and evaluation of workflow delays.

Data quality governance should include a correction process. Incorrect coordinates, duplicate events, misclassified damage, and incorrect repair status must be correctable without erasing the original event history. A correction record should preserve the previous value, new value, reason, and responsible actor. Data versioning also helps researchers reproduce past spatial summaries after records are updated. When publishing aggregate statistics, the system should report data coverage and avoid exposing sensitive exact locations where that could create a security or privacy risk.

The dashboard should make uncertainty visible. A low-quality GPS fix, uncertain class, stale observation, unverified repair, or unresolved duplicate should receive a specific status rather than being presented as equally reliable with a field-confirmed observation. This is a practical form of human-centered data governance: it helps operators distinguish what is known, what is inferred, and what still needs inspection.

## XII. SMART-CITY RELEVANCE, DEPLOYMENT, AND FUTURE WORK

India’s Smart Cities Mission seeks core infrastructure and a decent quality of life supported by smart solutions and sustainable urban environments [16]. A road-maintenance information layer supports this objective by improving the way condition observations are surfaced to municipal teams. Roadwatch also aligns with the intent of SDG 11, particularly target 11.2 on safe and accessible transport and target 11.3 on inclusive, sustainable urbanization [15]. Alignment should be understood as an intended contribution to local operational capacity, not as evidence that the application has independently improved safety outcomes.

A practical deployment path can be staged. The first stage standardizes the event schema and coordinates, documents the current manual workflow, and runs the dashboard with curated records. The second stage introduces an

authenticated API and database so that multiple operators see the same authoritative record. The third stage integrates a detector or inspection app, including model version, image evidence, timestamps, and GPS quality. The fourth stage adds duplicate grouping, segment-level rates, priority explanations, and audit exports. The final stage evaluates impact through a pilot with predefined baselines and review procedures.

The system can integrate with a municipal geographic information system, road inventory, work-order application, or complaint portal through stable APIs. Integration should rely on shared IDs and documented event semantics rather than direct cross-database assumptions. If an existing civic platform already owns asset identifiers or repair statuses, Roadwatch should link to that authoritative record and avoid creating conflicting copies. Data exchange can use a versioned JSON schema and idempotency keys to reduce duplicate ingestion.

Future computer-vision work should begin with a clear dataset policy. RDD2022 offers a relevant starting point because it includes Indian road imagery and multiple defect categories [2]. However, the local team should establish image acquisition conditions, annotation rules, inter-annotator review, route-based split boundaries, and a local holdout set. Data augmentation should reflect plausible changes in brightness, blur, perspective, and weather without introducing unrealistic artifacts. Class imbalance should be monitored at the image and instance levels, and a model should be selected based on operational false-positive and false-negative costs rather than a single headline metric.

Future geospatial analytics can add map-matched segments, repeat-condition detection, inspection coverage, crash or traffic exposure where appropriate and lawfully available, and time-to-repair distributions. Hotspot outputs should expose the bandwidth or neighborhood, date window, event count, denominator, and confidence/uncertainty context. Maintenance prioritization should remain policy-controlled, and a supervisor should be able to override a suggested ranking with an explicit reason. A future dashboard could compare planned versus actual completion dates and show repeat-repair rates by road segment.

Before scaling, the project should establish service-level expectations for map availability, event ingestion latency, database backup, restoration, browser compatibility, accessibility, and key rotation. If a public tile service becomes unreliable or its policies change, the system should be able to substitute an appropriate contracted provider or hosted tile infrastructure. A static page is an effective interface distribution method, but municipal operations require a governance plan for credentials, data access, retention, ownership, and incident response.

**TABLE III. STAGED DEPLOYMENT ROADMAP**

| Stage | Capability | Evidence required to exit stage |
|---|---|---|
| 1. Standardize | Canonical schema, coordinate rules, workflow definitions. | Schema tests; data dictionary; named owners. |
| 2. Persist | Authenticated API, database, audit history, concurrent users. | Integration tests; backup/restore; role tests. |
| 3. Connect sensing | Image/event ingestion, model version, GPS quality and evidence link. | Held-out detector metrics; location validation; error review. |
| 4. Add analytics | Duplicate grouping, road-segment rates, hotspots, explainable ranking. | Labeled duplicate pairs; sensitivity tests; expert review. |
| 5. Pilot and evaluate | Operational deployment with defined baselines and outcome measures. | Time-to-review/repair, reopen rate, coverage and user study. |

## XIII. CONCLUSION

This paper presented Roadwatch, a browser-based geospatial decision-support application for organizing road-defect records and supporting municipal maintenance triage. The system combines a Leaflet map, CARTO/OpenStreetMap-derived basemap, severity-coded markers, record summaries, time-range filtering, search, sorting, a linked detail drawer, and status-transition actions within one interface. A shared state model keeps map and table interactions aligned, while a structured event schema creates a stable boundary for additional data sources.

The paper’s contribution is the integration framework and its evaluation design. It distinguishes perception quality from geographic accuracy and workflow outcome, and defines reproducible methods for coordinate validation, spatiotemporal duplicate grouping, road-segment aggregation, hotspot discovery, and explainable priority scoring. The current 30-record registry provides descriptive evidence for the dashboard’s aggregation paths, while the functional test matrix documents core interaction behavior. These results do not substitute for a labeled model benchmark or a municipal impact evaluation; those studies are specified separately so that future claims can be supported by appropriate evidence.

Roadwatch offers a practical foundation for data-informed road maintenance in a smart-city context. The next research phase should connect a persistent event service, validate location provenance, benchmark a detector on held-out imagery including local conditions, measure duplicate and geospatial error, and evaluate whether the integrated workflow improves operational outcomes. A transparent and auditable design can turn detections into actionable infrastructure records while preserving human review and the uncertainty attached to the underlying evidence.

## ACKNOWLEDGMENT

The authors thank the faculty guide and project reviewers for their feedback on the system design and evaluation plan. Any funding, institutional support, dataset contribution, or

third-party assistance should be added here before submission.

## AUTHOR CONTRIBUTIONS

Author 1: [contribution statement]. Author 2: [contribution statement]. Author 3: [contribution statement]. Corresponding faculty author: [contribution statement]. The final statements should reflect the work actually carried out by each contributor and be approved by all listed authors.

## DATA AVAILABILITY

The application source code and the schema specification are available from the project repository maintained by the authors. Any redistribution of road imagery or external benchmark data must follow the corresponding dataset license and citation requirements. No private citizen imagery or municipal records should be made public without authorization.

## CONFLICT OF INTEREST

The authors should confirm and replace this statement before submission: “The authors declare no conflict of interest.” If any author has a relevant interest, relationship, sponsorship, or intellectual-property filing, it must be disclosed according to the target venue’s requirements.

## REFERENCES

[1] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, “Road damage detection and classification using deep neural networks with smartphone images,” Computer-Aided Civil and Infrastructure Engineering, vol. 33, no. 12, pp. 1127–1141, 2018, doi: 10.1111/mice.12387.

[2] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, “RDD2022: A multi-national image dataset for automatic road damage detection,” Geoscience Data Journal, vol. 11, no. 4, pp. 846–862, 2024, doi: 10.1002/gdj3.260.

[3] S. H. Abdelwahed, B. K. Sharobim, B. Wasfey, et al., “Advancements in real-time road damage detection: a comprehensive survey of methodologies and datasets,” Journal of Real-Time Image Processing, vol. 22, Art. no. 137, 2025, doi: 10.1007/s11554-025-01683-1.

[4] G. Singh, D. Bansal, S. Sofat, and N. Aggarwal, “Smart patrolling: An efficient road surface monitoring using smartphone sensors and crowdsourcing,” Pervasive and Mobile Computing, vol. 40, pp. 71–88, 2017, doi: 10.1016/j.pmcj.2017.06.002.

[5] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” in Advances in Neural Information Processing Systems, vol. 28, 2015.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, real-time object detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in Proc. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.324.

[8] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in Computer Vision – ECCV 2014, Lecture Notes in Computer Science, vol. 8693, pp. 740–755, 2014, doi: 10.1007/978-3-319-10602-1_48.

[9] A. Getis and J. K. Ord, “The analysis of spatial association by use of distance statistics,” Geographical Analysis, vol. 24, no. 3, pp. 189–206, 1992, doi: 10.1111/j.1538-4632.1992.tb00261.x.

[10] L. Anselin, “Local indicators of spatial association—LISA,” Geographical Analysis, vol. 27, no. 2, pp. 93–115, 1995, doi: 10.1111/j.1538-4632.1995.tb00338.x.

[11] P. A. P. Moran, “Notes on continuous stochastic phenomena,” Biometrika, vol. 37, nos. 1–2, pp. 17–23, 1950, doi: 10.1093/biomet/37.1-2.17.

[12] V. Agafonkin and Leaflet contributors, “Leaflet Quick Start Guide,” Leaflet documentation. [Online]. Available: https://leafletjs.com/examples/quick-start/. Accessed: Oct. 11, 2026.

[13] CARTO, “CARTO Basemaps API key and attribution.” [Online]. Available: https://carto.com/basemaps/apikey/ and https://carto.com/attribution/. Accessed: Oct. 11, 2026.

[14] OpenStreetMap Foundation, “Tile usage policy.” [Online]. Available: https://operations.osmfoundation.org/policies/tiles/. Accessed: Oct. 11, 2026.

[15] United Nations Department of Economic and Social Affairs, “Goal 11: Make cities and human settlements inclusive, safe, resilient and sustainable.” [Online]. Available: https://sdgs.un.org/goals/goal11. Accessed: Oct. 11, 2026.

[16] Ministry of Housing and Urban Affairs, Government of India, “Implementation of Smart Cities Mission,” Press Information Bureau, Dec. 11, 2023. [Online]. Available: https://www.pib.gov.in/Pressreleaseshare.aspx?PRID=1984980.

[17] R. Y. Wang and D. M. Strong, “Beyond accuracy: What data quality means to data consumers,” Journal of Management Information Systems, vol. 12, no. 4, pp. 5–33, 1996, doi: 10.1080/07421222.1996.11518099.

[18] Ultralytics, “YOLO performance metrics: precision, recall, F1 score and mean average precision.” [Online]. Available: https://docs.ultralytics.com/guides/yolo-performance-metrics/. Accessed: Oct. 11, 2026.

## APPENDIX A. RECOMMENDED EVENT JSON CONTRACT

The following JSON object illustrates the structure expected by an ingestion adapter. Field names can be adapted to an existing API, but the semantics and validation rules should remain stable. Model-specific outputs should be mapped into `damage_class`, `confidence`, and `evidence_ref`; the original model label and version should be retained for reproducibility.

```
{
 "schema_version": "1.0",
 "event_id": "RD-2026-000001",
 "observed_at": "2026-10-11T12:30:00Z",
 "received_at": "2026-10-11T12:30:02Z",
 "location": {
  "latitude": 13.0397,
  "longitude": 80.2442,
  "crs": "EPSG:4326",
  "horizontal_accuracy_m": 8.0,
  "road_segment_id": null,
  "validation_state": "needs_review"
 },
 "damage": {
  "class": "pothole",
  "severity": "critical",
  "confidence": 0.91,
  "model_name": "road-damage-detector",
  "model_version": "1.0.0"
 },
 "source": {
  "type": "vehicle_camera",
  "device_id": "device-identifier",
  "evidence_ref": "protected://evidence/item"
 },
 "workflow": {
  "status": "new",
  "assigned_crew_id": null,
  "priority_score": null,
  "status_history": []
 }
}
```

## APPENDIX B. RECOMMENDED VALIDATION AND REGRESSION TESTS

The following tests extend the interface checks with data-quality and service-level verification required for connected operation. A test report should include environment, browser, build identifier, dataset version, expected behavior, actual behavior, pass/fail, defect reference, and retest result.

**TABLE IV. EXTENDED TEST PLAN**

| Test group | Representative case | Acceptance condition |
|---|---|---|
| Schema | Missing ID, timestamp, severity, or invalid confidence | Event rejected or explicitly quarantined with a reason. |
| Geospatial | Latitude/longitude swapped or far outside area | Coordinate check fails and event is not silently plotted. |
| Road match | Valid coordinate near parallel carriageways | Candidate segment and match distance remain inspectable. |
| Deduplication | Repeated frames for one defect | Raw events preserved; grouping link is reviewable. |
| Status history | Two users update the same record | Conflict policy prevents silent overwrite; audit trail persists. |
| Persistence | Refresh and open from a second session | Committed state and record ID remain consistent. |
| Security | Use API without required role or invalid token | Request denied and logged. |
| Tile resilience | Provider returns timeout / authorization error | Application remains usable and communicates map-layer error. |
| Backup / restore | Restore a database snapshot | Record counts, IDs, and state history pass integrity checks. |
| Analytics reproducibility | Recompute rates using same data version and parameters | Same results and metadata are produced. |

## APPENDIX C. PROPOSED PRIORITY COMPONENT DEFINITIONS

Table V lists illustrative component definitions for the priority score. The ranges and transformations should be approved by the road authority before use. The paper does not prescribe universal weights because road hierarchy, safety exposure, repair resources, and local policy differ between municipalities.

**TABLE V. INTERPRETABLE PRIORITY FEATURES**

| Feature | Example normalization | Interpretation |
|---|---|---|
| Severity S | Ordinal class mapped to [0,1] | Engineering assessment of defect seriousness. |
| Exposure E | Road class or approved traffic measure, scaled [0,1] | Potential exposure of road users or critical access. |
| Recurrence R | Capped unique observations in a defined period | Persistent or repeatedly observed condition. |
| Age A | min(waiting time / policy horizon, 1) | Length of time awaiting review or maintenance. |
| Evidence quality U | Separate class or confidence band | Need for human verification; not automatically urgency. |